

Department of Computer Science and Engineering, IIT Palakkad

CS 2030 Data Structures and Algorithms

Quiz 2 (3 Apr 2024)

Time: 8:00 hrs to 8:50 hrs

Points: 25

27

Name: ... Krithika

Roll No:

Instructions

1. This quiz-booklet has 8 pages containing 4 questions. Please get a new booklet if any question/page is missing.
2. Write your **name** and **roll number** in the space above.
3. Answer each question in the corresponding space provided. **Additional sheets may be used only for rough work and will not be evaluated.**
4. Write your answers neatly in blue or black ink. Do not use pencil or red ink.
5. Do not keep any books, papers, mobile phones or other smart gadgets with you during the exam.
6. All answers require adequate justifications.

1. (a) Define min-heap. How will you represent a min-heap using an array?

4

A min heap is a nearly complete binary tree s.t key at a node is not greater than the keys at its children

Array A $A[1] \rightarrow \text{root}$ 1 mark

1 mark

parent of $A[i]$
stored in $A[\lfloor \frac{i}{2} \rfloor]$

$A[i]$
/ \
 $A[2i]$ $A[2i+1]$
left(i) right(i)

1 mark

1 mark

- (b) Show that the root of any subtree of a min-heap contains the smallest value occurring anywhere in that subtree. Where can you find the smallest three elements in a min-heap containing $n \geq 3$ distinct elements?

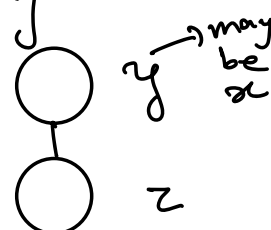
4

Suppose $\exists T_x$ subtree rooted at x that violates the claimed property. Then $\exists$ a descendant z of x s.t $x.\text{key} > z.\text{key}$. Among all such z choose one closest to x .

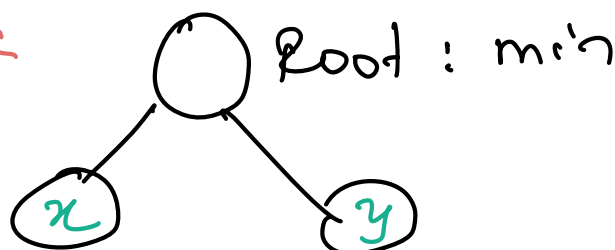
Let y be the parent of z

Then $z.\text{key} < x.\text{key} \leq y.\text{key}$.

implying that min heap prop at y is violated leading to a contradiction. 3 marks



1 mark



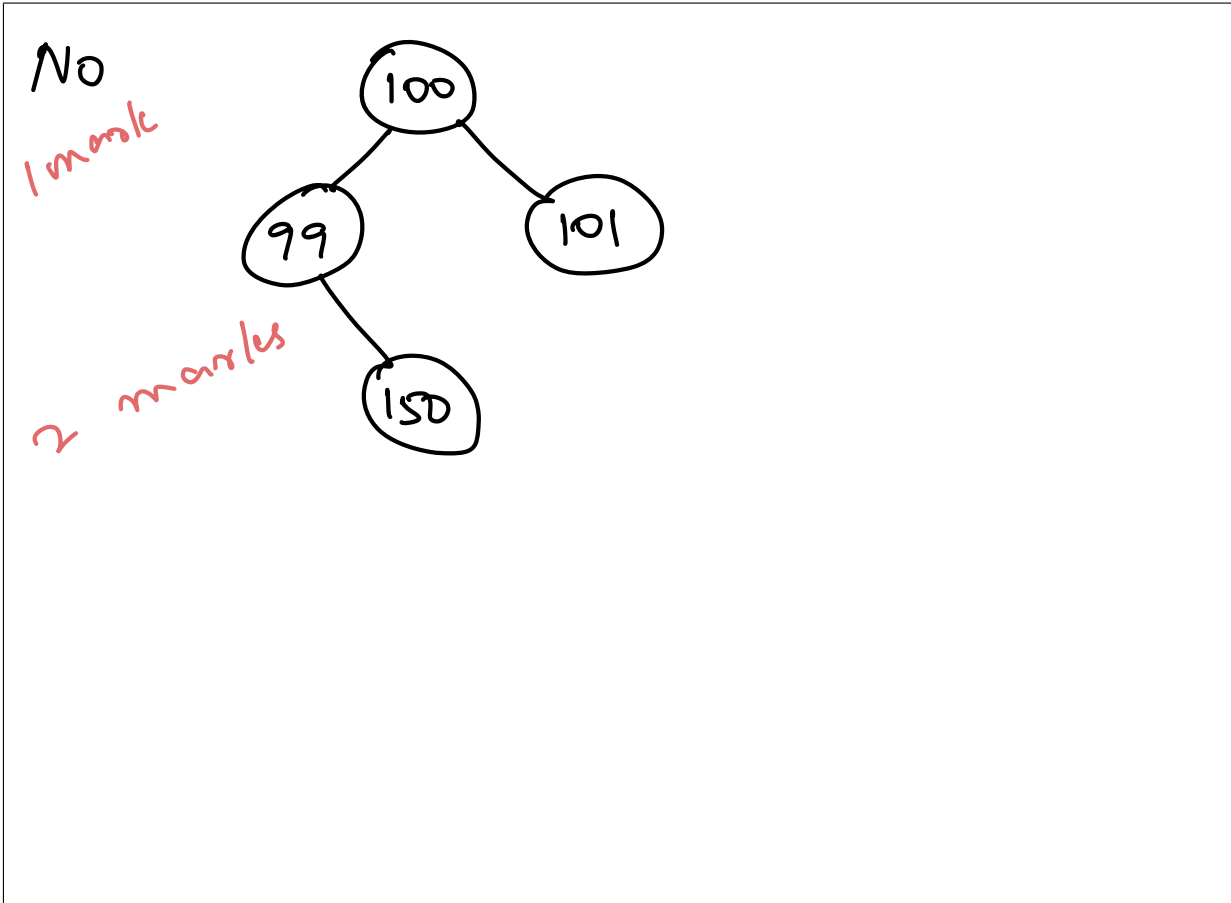
Sec min: x or y

If x is sec min, third min is y or some ele in T_x . Similarly in other case

2. Recall that a binary search tree is a binary tree such that for each node x , all nodes stored in the left subtree of x have key values lesser than or equal to $x.key$ and all nodes stored in the right subtree of x have key values greater than or equal to $x.key$.

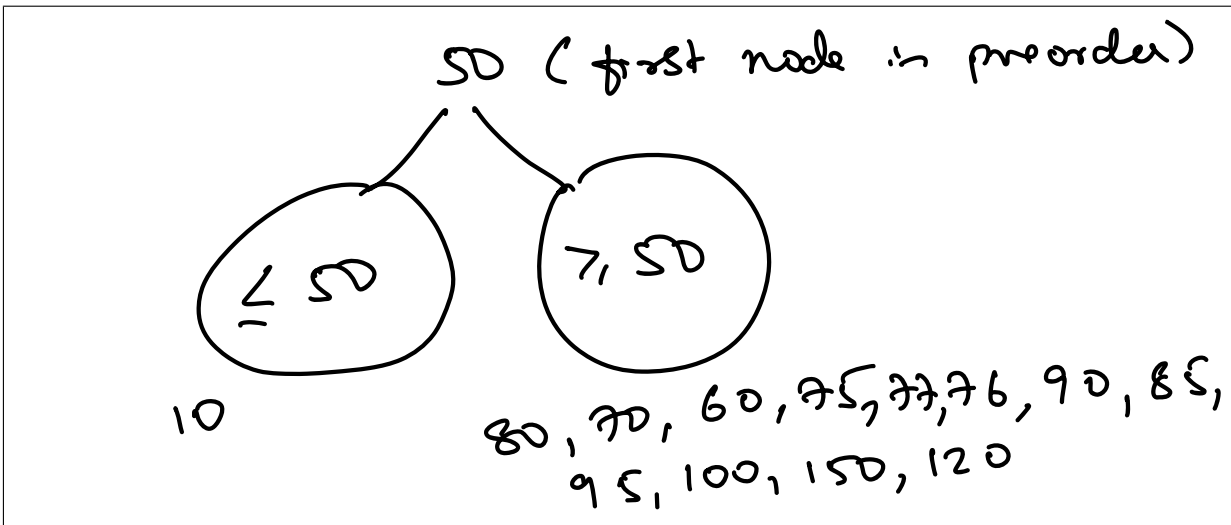
- (a) Consider the following modified definition: a binary search tree is a binary tree such that for each node x , the left child of x (if it exists) has key value lesser than or equal to $x.key$ and the right child of x (if it exists) has key value greater than or equal to $x.key$. Is this definition equivalent to the actual one? Justify.

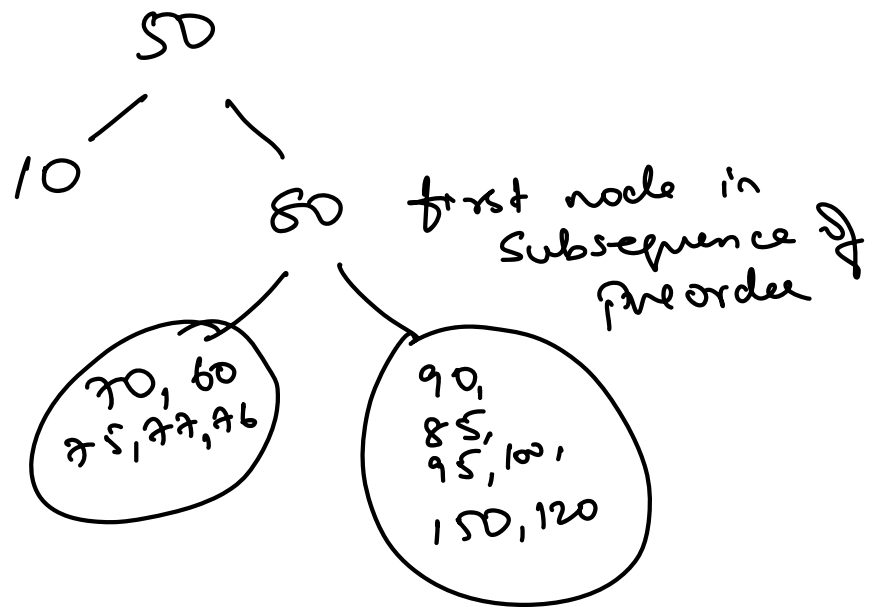
3



- (b) Draw the binary search tree whose preorder traversal sequence is 50, 10, 80, 70, 60, 75, 77, 76, 90, 85, 95, 100, 150, 120. How did you arrive at this tree from the sequence? Give the inorder and postorder traversal sequences of this tree.

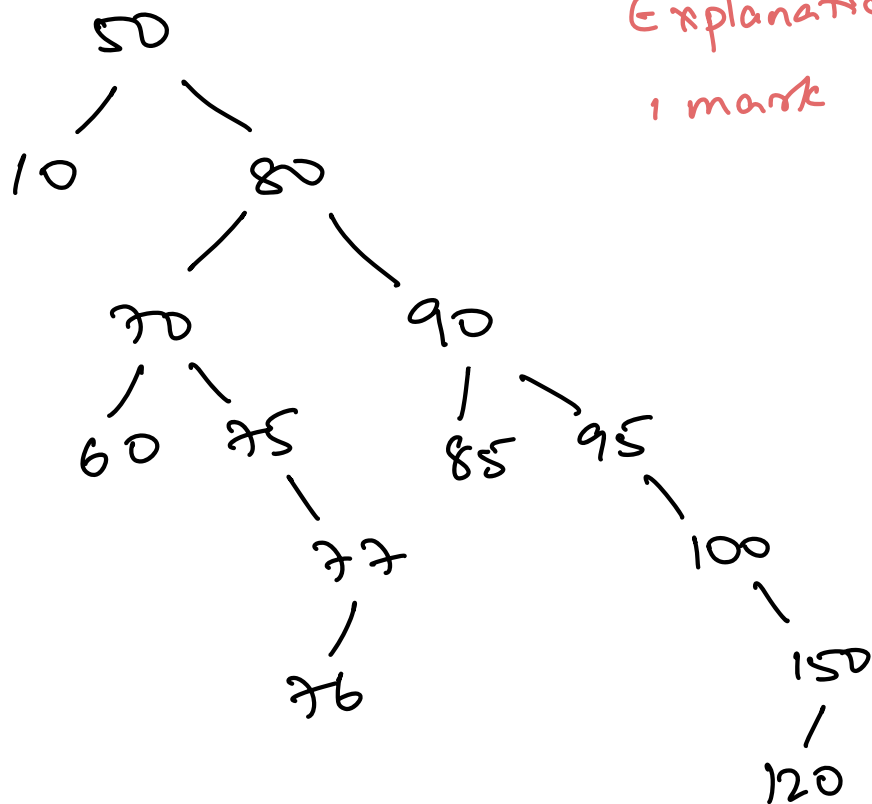
4





1 mark

Explanation
1 mark



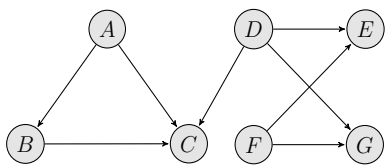
1 mark

Inorder: 10, 50, 60, 70, 75, 76, 77, 80, 85, 90, 95, 100, 120, 150

1 mark

Postorder: 10, 60, 76, 77, 75, 70, 85, 120, 150, 100, 95, 90, 80, 50

3. Write the adjacency matrix for the below directed acyclic graph. 2 marks



	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	0	0	1	0	0	0	0
C	0	0	0	0	0	0	0
D	0	0	1	0	1	0	1
E	0	0	0	0	0	0	0
F	0	0	0	0	1	0	1
G	0	0	0	0	0	0	0

4. Consider a chess tournament of n players $P_1, P_2, \dots, P_n$ in which each player plays with every other player exactly once and no game results in a draw. The results of the matches are given as a boolean matrix M where $M[i][j]$ is 1 if player P_i wins against player P_j and 0 otherwise. The objective is to rank the players such that for any two players X and Y , X receives a rank higher than that of Y if and only if X won in the match against Y .

- (a) Give an example where no such ranking is possible. 2
- (b) Model this problem as a graph-theoretic problem and state the necessary and sufficient conditions for the desired ranking to exist. 4
- (c) Given input M with the guarantee that the desired ranking exists, describe an algorithm that produces the desired ranking. State the key theorems used in proving the correctness of your algorithm. Also, analyze the running time of the algorithm. 4

(a) $P_1 \xrightarrow{\text{beats}} P_2 \xrightarrow{\text{beats}} P_3$

$\curvearrowright$ beats

2 marks

(b) vertices : Players

Edges $P_i \rightarrow P_j$ if P_i wins against P_j

1+1 marks

The desired ranking exists if (*)
and only if the corres graph
 G has a topo order 2 marks

(c) Input : M $n \times n$ matrix

Step 1 : Construct corres graph G_M
 $O(n^2)$ time as n^2 entries to look at

Step 2 : Run DFS on G_M +
output the vertices in decreasing
order of their finish times

$O(n^2)$ time as $m = \binom{n}{2}$

G_M is acyclic iff DFS on G_M
reveals no back edge
only back edges (u, v) have starting time,
finish times as $\frac{v}{u} + (*)$ Prop

Algorithm : 2 marks

$\Rightarrow$ Dec order of
FTs is topo
order

Running Time Justification : 1 mark

Key Theorem : 1 mark

